



Lesson 6.4 — Dynamic Content + Cache Components

Now we're hitting the **really important part**.

You already know that some data is cacheable and some is request/user-specific. The question is:

What happens when a page contains BOTH cached and dynamic content?

This is where **Cache Components + Suspense + PPR** come together. 🔥

1. Our example

Imagine a product page:

```
export default function ProductPage() {  
  return (  
    <>  
    <Header />  
    <Product />  
    <Recommendations />  
    <Cart />  
    </>  
  )  
}
```

And:

Header
↓
static

Product
↓
cacheable

Recommendations
↓
cacheable

Cart
↓
user-specific / dynamic

Visually:



The interesting question is:

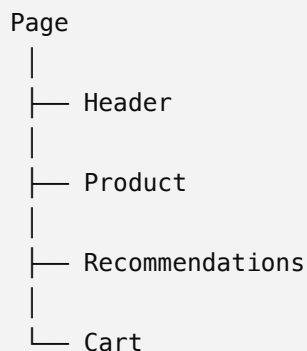
Does the dynamic `Cart` force the entire page to be dynamically rendered?

With the modern Cache Components model, **not necessarily**.

2. Think in terms of boundaries

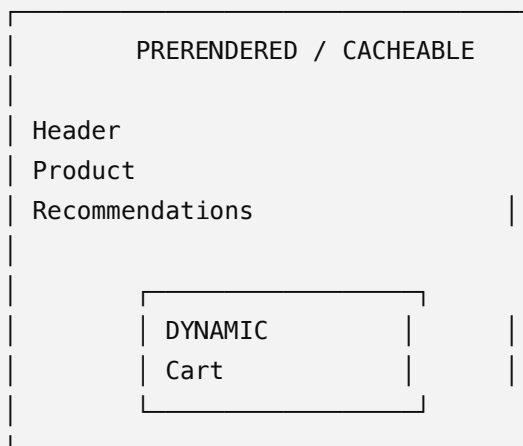
This is the key idea.

We can have:



and establish boundaries around the parts that can be cached/prerendered.

Conceptually:



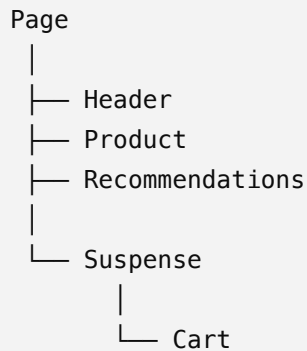
That dynamic portion is sometimes described as a **dynamic hole** in the prerendered output.

3. Where Suspense comes in

To make this separation explicit, we can use:

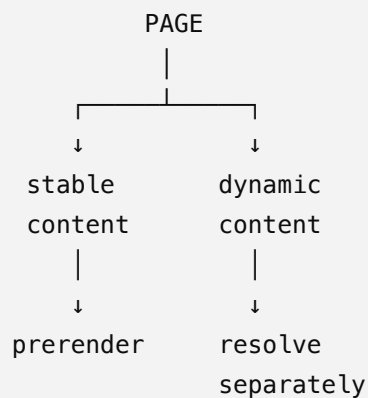
```
<Suspense fallback={<CartSkeleton />}>  
  <Cart />  
</Suspense>
```

Now:



The Suspense boundary gives Next.js a place where dynamic/async work can be separated from the surrounding content.

Think:



4. Why is this useful?

Imagine:

```
Product data → 50ms  
Recommendations → 100ms  
Cart → 800ms
```

Without granular handling, the slow cart could hold up the entire page.

With a boundary:

Browser receives:

Header
Product
Recommendations
Cart Skeleton

Then:

Cart
↓
finishes
↓
actual Cart UI

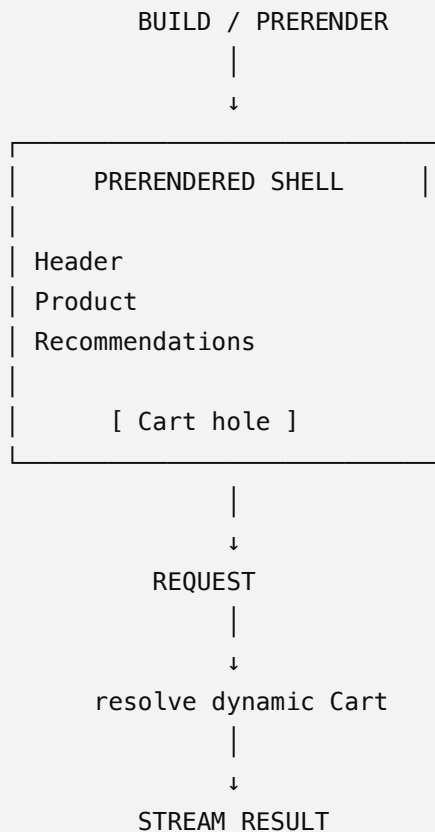
So the user can start seeing useful content earlier.

That's one of the major benefits of **streaming + Suspense**.

5. PPR mental model

Now combine this with Partial Prerendering.

You can think of the result as:



So the important idea is:

The static/cacheable parts don't have to wait for every dynamic part.

6. But here's an important correction to a common misconception

Don't think:

"Suspense makes something dynamic."

✗ No.

Suspense itself doesn't mean:

Suspense = dynamic



Instead, Suspense provides a **boundary for asynchronous rendering/streaming**.

The thing inside the boundary could be:

- dynamic
- asynchronous
- slow
- or simply something you want to stream separately.

So:

Suspense



boundary

NOT

Suspense



automatically dynamic



That's an important interview distinction.

7. Now let's bring `cookies()` into it

Suppose:

```
async function Cart() {  
  const cookieStore = await cookies()  
  
  const userId = cookieStore.get("userId")  
  
  return getCart(userId)  
}
```

The cart depends on the current request/user.

Conceptually:

```
Request A  
  ↓  
cookie = userA  
  ↓  
Cart A  
  
Request B  
  ↓  
cookie = userB  
  ↓  
Cart B
```



So we don't want to treat the cart as universally shared cached content.

Instead:

```
Product  
  ↓  
cacheable  
  
Cart  
  ↓  
request-dependent
```



And the boundary keeps those concerns separate.

8. Here's the BIG connection to what you learned earlier

Earlier, you learned:

```
cookies()  
headers()
```



```
searchParams
no-store
```

can make work request-dependent.

You also learned:

```
static rendering
dynamic rendering
```



Now Cache Components gives us a more granular mental model.

Instead of:

```
Dynamic thing exists
  ↓
Entire page = dynamic
```



think:

```
Dynamic thing
  ↓
dynamic boundary
  ↓
other cacheable/prerenderable work can remain cacheable
```



That's the conceptual evolution.

9. Let's trace the whole request

Suppose:

```
/products/123
```



contains:

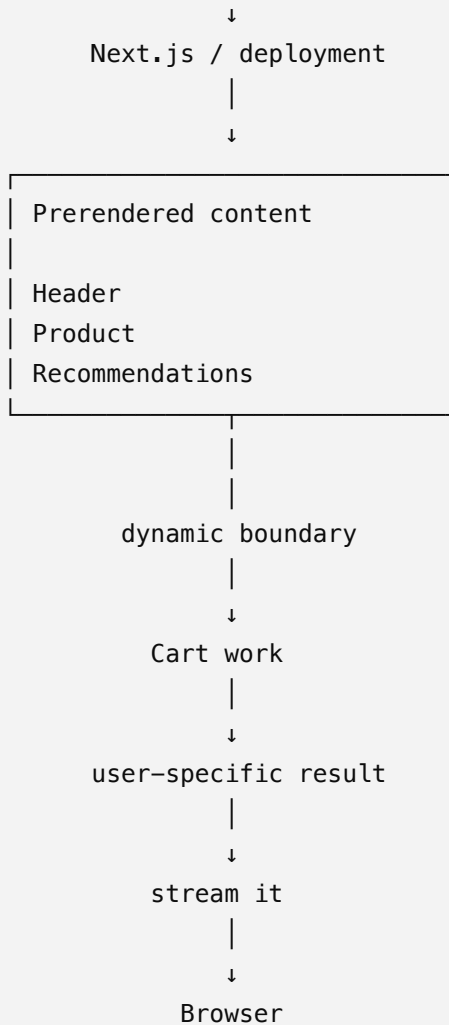
```
Header      → static
Product     → cached
Recommendations → cached
Cart        → dynamic
```



A simplified mental model:

```
REQUEST
|
```





10. Where does "use cache" fit?

Suppose:

```

async function Product() {
  "use cache"

  cacheLife("hours")
  cacheTag("product:123")

  return getProduct()
}
  
```

Now:

```

Product
|
├── "use cache"
|
  
```



```

├─ cacheLife()
├─
└─ cacheTag()

```

While:

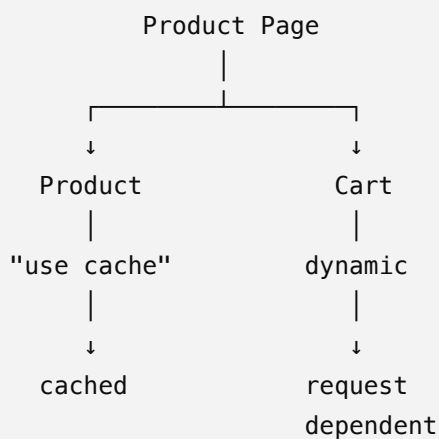
```

async function Cart() {
  const user = await getCurrentUser()
  return getCart(user.id)
}

```

is dynamic.

So:



11. One subtle thing: cached component ≠ cached HTML

This is **very** important.

If you say:

```
"use cache"
```

don't automatically translate that into:

```

"use cache"
  ↓
HTML file stored somewhere

```

That's too simplistic.

The cacheable computation/rendering result participates in the **Cache Components system**.

So distinguish:

Cached computation/component result



from:

Rendered route / HTML response



and from:

Browser Router Cache



and from:

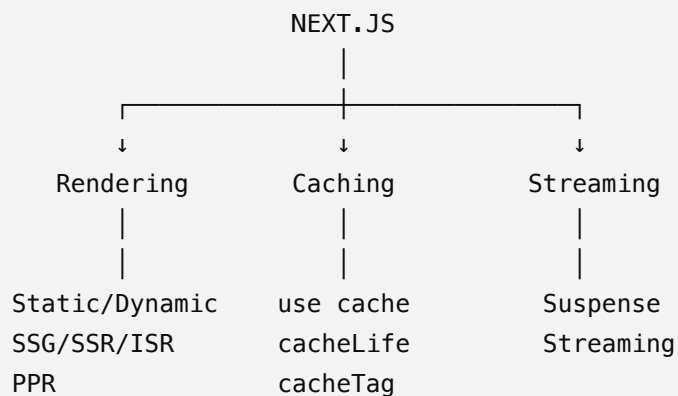
CDN HTTP cache



These are different layers.

12. Our complete mental model 🧠

You've now got:



And these interact:

```

"use cache"
+
cacheLife()
+
cacheTag()
+
Suspense
  
```



+
dynamic content
↓
Cache Components + PPR

The interview version

If an interviewer asks:

"How can Next.js handle a page with both static and dynamic content?"

You can say:

"With the modern Cache Components model, cacheable portions can be marked with `use cache`, while request-dependent work can remain dynamic. Suspense boundaries can isolate asynchronous or dynamic portions, allowing Next.js to prerender or cache the stable parts and stream the dynamic portions when they become available."

That's a very solid answer. 🔥

Next: 6.5 — Cache Components vs the old caching model

This is the piece I really want you to learn next because you've already spent a lot of time mastering:

Data Cache
Full Route Cache
Router Cache



We'll put them side-by-side with:

"use cache"
cacheLife()
cacheTag()
PPR



and answer the big question:

"Did Next.js completely replace Data Cache and Full Route Cache, or do they still exist—and how do the models relate?"

That'll prevent one of the biggest sources of confusion when reading modern Next.js documentation. 😎🔥



and also how does Cached computation/component result look? and cacheLife hours or minutes means what exact time it gets invalidated and how it gets expired and how fresh data comes ...explain with real code scenario...and bcoz of this cached component now if one component or api is dynamicly rendered before now entire route doesnt need to do like that..

Yes — **this is exactly the right doubt to stop and clarify.** 😊🔥 And I want to slightly sharpen one thing from my previous explanation because the distinction matters.

You are asking **three connected questions**:

1. What does a **cached computation/component result** physically/logically look like?
2. If I say `cacheLife("hours")` , **when exactly does it become stale/expire, and where does fresh data come from?**
3. Does Cache Components mean **one dynamic component no longer automatically makes the whole route dynamic?**

The answer to #3 is **yes, that's the major mental shift** — but let's understand *why* rather than memorize it.

1. First: what is a "cached computation result"?

Take real code:

🔗 TypeScript

```
async function Product({ id }: { id: string }) {  
  "use cache"  
  
  const product = await db.product.findUnique({  
    where: { id },  
  })  
  
  return (  
    <div>  
      <h1>{product.name}</h1>  
      <p>₹{product.price}</p>  
    </div>  
  )  
}
```



```
)  
}
```

Suppose:

```
/Product/123
```



is requested.

The first time:

```
Product(123)
```

```
↓
```

```
"use cache"
```

```
↓
```

```
DB query
```

```
↓
```

```
{  
  id: 123,  
  name: "iPhone",  
  price: 79999  
}
```

```
↓
```

```
React renders component
```



The important thing is:

Next.js can retain the result of that cached computation so it doesn't have to perform the same server work again until the cache policy says otherwise.

Think of it conceptually as:

```
CACHE ENTRY
```

```
Key / identity:
```

```
  Product + id=123
```

```
Result:
```

```
  rendered/computed result for Product(123)
```

```
Metadata:
```

```
  lifetime information
```

```
  tags
```



It is **not simply** "a `.html` file sitting in a cache."

That's why I was emphasizing earlier:

"use cache"



cached computation/component result



is different from:

Full HTML response



and different from:

Browser Router Cache



The exact internal representation is an implementation detail; for your mental model, think **"memoized server-side result + cache metadata."**

2. Let's make it more concrete

Suppose:

</> TypeScript



```
async function getProduct(id: string) {  
  "use cache"  
  
  return db.product.findUnique({  
    where: { id },  
  })  
}
```

Request:

/product/123



First request:

Browser



Next.js



getProduct(123)



CACHE MISS



Database



```
Product 123
  ↓
store cached result
  ↓
render page
  ↓
Browser
```

Now second request:

```
Browser
  ↓
Next.js
  ↓
getProduct(123)
  ↓
CACHE HIT ✅
  ↓
cached Product 123
  ↓
render/use result
  ↓
Browser
```

So the expensive work:

```
database query
```

doesn't necessarily have to happen again.

3. Now let's add `cacheLife()`

Suppose:

```
</> TypeScript

async function getProduct(id: string) {
  "use cache"

  cacheLife("hours")

  return db.product.findUnique({
    where: { id },
  })
}
```

Here's where we need to be precise.

"hours" does NOT mean:

"Exactly 60 minutes after the first request, delete this cache entry."

✗ Don't build that mental model.

A cache life profile defines **freshness/staleness/expiration behavior**, rather than being simply "delete after N minutes."

The predefined profiles are designed around those lifecycle concepts.

4. Let's use a simpler custom example

Imagine we have a policy conceptually like:

```
stale after: 1 hour
revalidate after: ...
expire after: ...
```



Don't focus on the exact numbers yet — focus on the lifecycle.

At:

10:00



the cache is created.

10:00



CACHE ENTRY CREATED



Then:

10:00 ————— 11:00
FRESH



If someone requests it at:

10:30



Next.js can use the cached result.

10:30



```
CACHE HIT
↓
still fresh
↓
return cached result
```

5. Then it becomes stale

At some point:

11:00



the cached result is no longer considered **fresh**.

Important:

STALE $\neq$ immediately deleted



Think:

```
Fresh
↓
"this cached result is recent enough"

Stale
↓
"this cached result is old enough that
we should obtain fresh content according
to the cache policy"
```



So don't imagine a tiny garbage collector coming along at exactly 11:00 and deleting it.



The cache lifecycle determines how Next.js treats that entry when requests happen.

6. So where does the fresh data come from?



This is the part you were specifically asking about.

Suppose our database changed:

```
10:00
Product price = ₹79,999
```



11:10

Admin changes price

Product price = ₹74,999

But our cached result still says:

₹79,999



A later request needs fresh information.

The flow becomes conceptually:

```
Request
  ↓
cached result
  ↓
STALE
  ↓
Next.js performs the cached computation again
  ↓
database
  ↓
₹74,999
  ↓
new result
  ↓
cache updated
```



So:

Fresh data comes from rerunning the underlying computation.

For example:

```
</> TypeScript
const product = await db.product.findUnique(...)
```



runs again.

Or if your cached function calls:

```
</> TypeScript
await fetch("https://api.example.com/products/123")
```



that underlying operation is performed again according to its own behavior.

7. This is VERY different from "cache magically knows the DB changed"

A cache doesn't automatically know:

Database changed!



unless you explicitly connect the mutation to cache invalidation/revalidation.

That's why we learned:

cacheTag()



and:

revalidateTag()
updateTag()



For example:

`</>` TypeScript



```
async function getProduct(id: string) {  
  "use cache"  
  
  cacheLife("hours")  
  cacheTag(`product:${id}`)  
  
  return db.product.findUnique({  
    where: { id },  
  })  
}
```

Then after updating Product 123:

`</>` TypeScript



```
await updateProductInDatabase()  
  
revalidateTag("product:123")
```

Now you've explicitly told Next.js:

"Hey, anything associated with product:123
needs to be refreshed according to the API's semantics."



This is why **time-based freshness** and **explicit invalidation** are complementary.

8. Two ways cached data becomes outdated

Think:

A. Time-based

```
cacheLife()  
  ↓  
time passes  
  ↓  
fresh → stale/expire  
  ↓  
recompute when needed
```



B. Explicit invalidation

```
Database mutation  
  ↓  
revalidateTag()  
  ↓  
associated cache becomes stale / is revalidated  
  ↓  
fresh computation
```



So:

```
CACHE LIFETIME  
  +  
CACHE INVALIDATION  
  ↓  
fresh cached content
```



9. Now your BIG question: does dynamic content still make the whole route dynamic?

This is the important modern shift: not necessarily. 🔥

Imagine:

```
</> TypeScript  
  
export default async function ProductPage() {  
  return (  
    <>
```



```
<Header />

<Product />

<Recommendations />

<Suspense fallback={<CartSkeleton />}>
  <Cart />
</Suspense>
</>
)
```

Suppose:

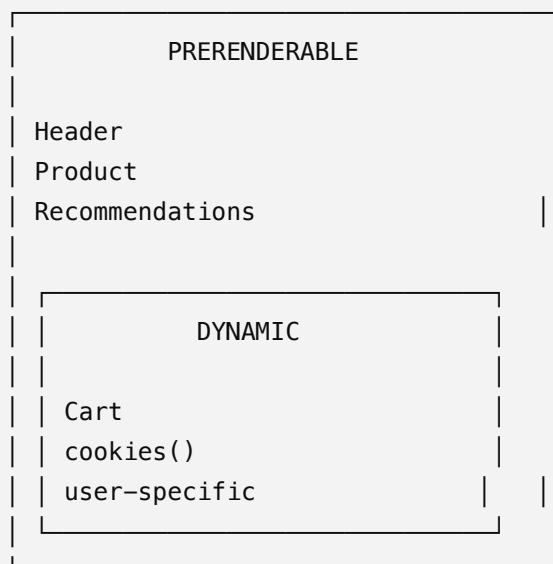
Header
↓
static

Product
↓
"use cache"

Recommendations
↓
"use cache"

Cart
↓
uses cookies()
↓
dynamic

Conceptually:



So the existence of a dynamic component doesn't automatically mean every other component has to be dynamically generated for every request.

That's the power of the Cache Components/PPR model.

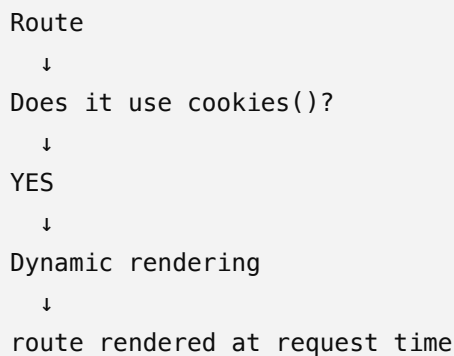
Next.js 16 introduced Cache Components as a programming model built around `use cache` and Partial Prerendering.  Next.js

10. Compare old mental model vs modern mental model

This is where I want you to update your brain. 🧠

Earlier model we learned

You might have reasoned:

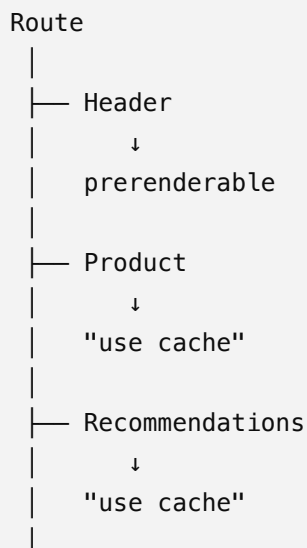


```
Route
  ↓
Does it use cookies()?
  ↓
YES
  ↓
Dynamic rendering
  ↓
route rendered at request time
```

That model is useful for understanding the older App Router caching/rendering behavior.

Modern Cache Components model

We can reason more granularly:



```
Route
├── Header
│   └── prerenderable
│       └── "use cache"
├── Product
│   └── "use cache"
└── Recommendations
    └── "use cache"
```

```
└─ Cart
  ↓
  dynamic
  ↓
  cookies()
```

Now:

```
Prerendered shell
+
dynamic portion
↓
Partial Prerendering
```



That's the big upgrade.

11. Real example with your Web Performance Analyzer

Let's make this relevant to **your project**. 😎

Imagine your page:

```
/analyze/google.com
```



contains:

```
Performance Analyzer
Google.com
Performance Score: 97
Accessibility: 94
SEO: 100
[Analyze Again]
User's analysis history
```



Suppose PageSpeed results don't need to be regenerated every second.

You could conceptually have:

```
</> TypeScript
```



```

async function getPageSpeedResult(url: string) {
  "use cache"

  cacheLife("hours")
  cacheTag(`pagespeed:${url}`)

  return runPageSpeedAPI(url)
}

```

Then:

```

First request
  ↓
PageSpeed API
  ↓
expensive
  ↓
cache result

```



Later:

```

Request for google.com
  ↓
cache hit
  ↓
reuse result

```



But user-specific history:

```

</> TypeScript

async function History() {
  const user = await getCurrentUser()

  return getUserHistory(user.id)
}

```



is dynamic.

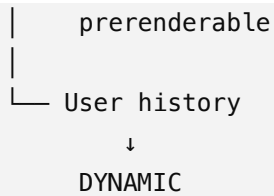
So:

```

/analyze/google.com
|
|— PageSpeed result
|   ↓
|   CACHE
|
|— Score UI
|   ↓
|

```





That is **exactly** the kind of scenario where this mental model becomes useful.

12. One subtle correction: "component is cached" doesn't mean React freezes it forever

This is another important distinction.

Suppose:

```
</> TypeScript
```



```
<Product />
```

is cacheable.

It doesn't mean:

```
Product
  ↓
render once
  ↓
never changes
```



Instead:

```
Product computation
  ↓
cache result
  ↓
reuse while valid
  ↓
lifetime/invalidation event
  ↓
recompute
  ↓
new cached result
```



So:

CACHE
≠
PERMANENT



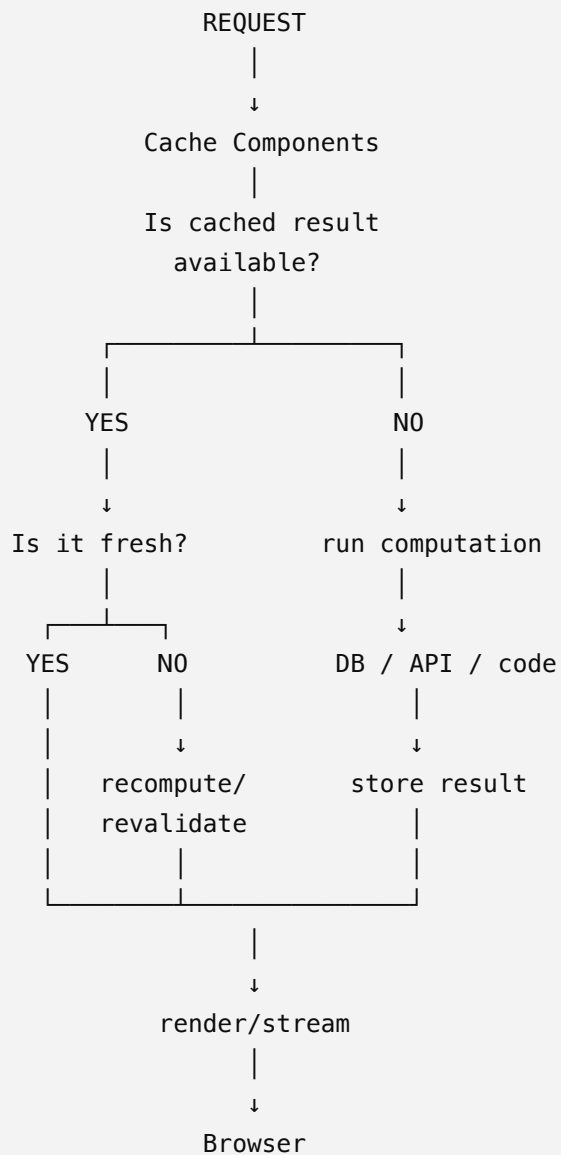
It's:

CACHE
=
REUSE UNTIL CACHE POLICY SAYS OTHERWISE



13. And here's the full lifecycle

Let's put **everything** together.



And meanwhile:

cacheLife()

↓

controls lifecycle

cacheTag()

↓

identifies related cached result

revalidateTag()/updateTag()

↓

explicitly triggers invalidation/revalidation behavior



14. One VERY important distinction about "exact time"

You asked:

"cacheLife("hours") means what exact time it gets invalidated?"

Don't think:

10:37:12

+

exactly 1 hour

↓

11:37:12

↓

DELETE



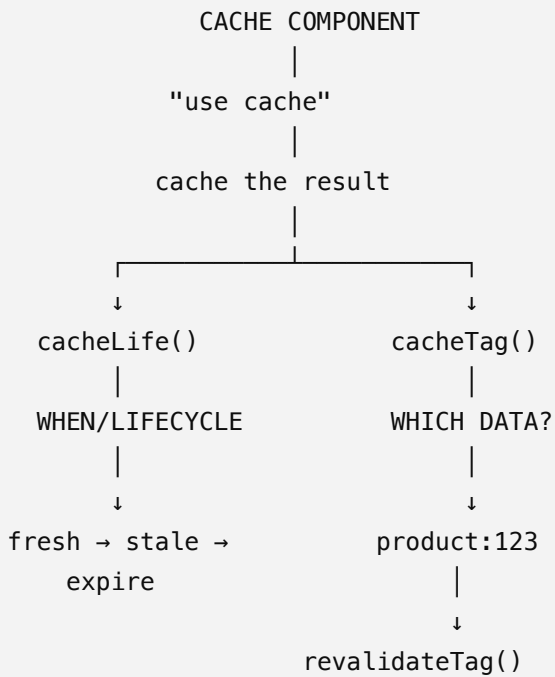
That's **not the right abstraction**.

The cache profile defines lifecycle semantics — including freshness and expiration — and Next.js handles when cached content is considered fresh/stale/expired and when it needs to be regenerated. The exact predefined profile values and behavior should be treated as framework configuration, not as a promise that a timer physically deletes the entry at an exact second.

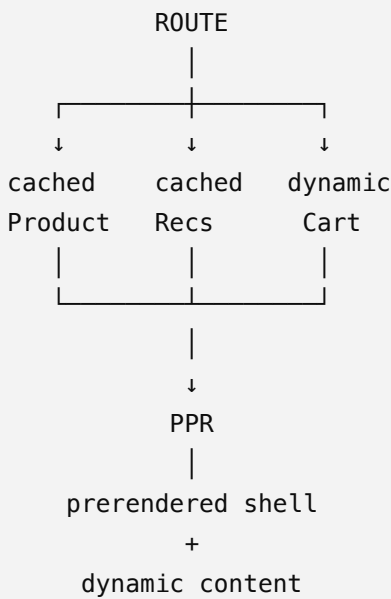
And **explicit invalidation can cause it to become outdated before its normal lifetime**.

15. Your final mental model 🧠🔥

I want you to remember this:



And separately:



So your statement is basically correct:

"Because of Cache Components, if one component/API is dynamic, the entire route doesn't necessarily have to be dynamically rendered anymore."

✅ Yes — that's the key conceptual shift.

But say it more precisely:

"With Cache Components and Partial Prerendering, a route can contain separately cached/prerenderable and dynamic portions. A dynamic portion does

not necessarily force the entire route's useful output to be regenerated dynamically for every request."

That's the interview-quality version. 100

And this is exactly why Next.js's newer caching model was introduced: to get the benefits of prerendering while still supporting dynamic/request-specific content.

 Next.js +1

Next, we should do one complete request from `GET /product/123` → cache lookup → prerendered shell → dynamic hole → browser, with actual code and timestamps.
That will make the whole thing click. 🔥





Yes — this is the **exact confusion to clear up**. 🤔 Once you understand this, Server Components become much less magical.

1. "If Server Component code isn't in the browser, how does it show on screen?"

Because **the browser doesn't need the Server Component code to display its result**.

Think about a restaurant 🍕 :

```
Chef (server)
  ↓
uses recipe + ingredients
  ↓
prepares pizza
  ↓
waiter delivers pizza
  ↓
Customer sees pizza
```



The customer doesn't need the **recipe** or the **kitchen** to see the pizza.

Same idea:

```
SERVER
Server Component code
  ↓
executes
  ↓
produces result
  ↓
HTML / RSC payload
  ↓
BROWSER
  ↓
screen
```



2. Concrete example

You write:

```
</> TypeScript

// app/products/page.tsx

export default async function ProductsPage() {
  const products = await db.product.findMany();

  return (
    <main>
      <h1>Products</h1>

      {products.map(product => (
        <div key={product.id}>
          {product.name}
        </div>
      ))}
    </main>
  );
}
```



```
    </main>
  );
}
```

This is a **Server Component**.

The browser does NOT need:

- ✗ ProductsPage.tsx
- ✗ db.product.findMany()
- ✗ database credentials
- ✗ database connection



The server executes:

```
ProductsPage()
  ↓
DB query
  ↓
React renders result
```



Suppose the result is:

```
Products:
iPhone
MacBook
AirPods
```



The browser can receive/render markup representing that result:

 HTML



```
<main>
  <h1>Products</h1>
  <div>iPhone</div>
  <div>MacBook</div>
  <div>AirPods</div>
</main>
```

So the browser says:

"Cool, I have markup. I'll paint this on the screen."

It doesn't need to know **how the server produced it**.

3. Then what is RSC doing?

Here's the important distinction:

Server Component CODE

↓

runs on server

↓

Server Component RESULT

↓

RSC payload

↓

browser React



The browser receives the **result/representation**, not the original server implementation.

That's why:



Browser:

"Let me execute ProductsPage.tsx"

vs



Browser:

"I received the result of ProductsPage."



4. Can you SEE Router Cache in your browser? 🙄

Yes — but there's an important caveat.

You won't normally open DevTools and see:

Router Cache

/products/123

/products/456



as a nice readable cache panel.

Next.js doesn't expose Router Cache as a simple Chrome DevTools storage bucket like:

Application

├ Local Storage

├ Session Storage

├ IndexedDB

└ Router Cache ← **X** not normally here



Router Cache is an **internal client-side cache maintained by the Next.js App Router**, not ordinary `localStorage`.

5. But you CAN inspect evidence of it 🔍

This is actually a great experiment for you.

Open your Next.js app in Chrome.

Then:

Chrome DevTools



Network



Navigate using:

</> TypeScript

```
<Link href="/products/123">
```



rather than:

HTML

```
<a href="/products/123">
```



You can inspect the network requests and see requests associated with the App Router/RSC navigation.

Depending on your Next.js version and dev/prod mode, you'll see RSC-related request/response information.

You may encounter headers such as:

RSC

Next-Router-State-Tree

Next-Router-Prefetch



and responses associated with the RSC protocol.

The exact presentation varies between development/production and Next.js versions.

6. Here's a really good experiment

Make two pages:

```
/products/123  
/products/456
```



and navigate:

```
123  
↓  
456  
↓  
123
```



using:

```
</> TypeScript  
  
<Link href="/products/123">
```



Watch:

```
DevTools  
→ Network
```



You'll be able to observe the App Router making RSC/navigation requests.

Then try:

```
/products/123
```



→

```
/products/456
```



→

```
/products/123
```



The Router Cache can allow previously received route information to be reused according to its current caching behavior.

7. Can you visually see the cached Server Component?

This needs a careful distinction.

✗ You generally won't see:

Next.js Server Cache

Product component
cached result

inside Chrome DevTools.

Why?

Because that cache is **on the server**, not in your browser.

Remember:

"use cache"

↓

SERVER-SIDE CACHE

while:

Router Cache

↓

CLIENT-SIDE

They're different.

8. So where can you inspect each one?

Thing	Where it lives	Can you inspect it?
Server Component source	Server/build	Not as browser JS
"use cache" result	Server-side cache	Not directly through normal browser DevTools
RSC response	Network	✔ Yes
HTML	Network → document response	✔ Yes

Thing	Where it lives	Can you inspect it?
Client JS	Network/Sources	✅ Yes
Router Cache	Next.js client runtime	⚠️ Not as a normal DevTools storage panel
localStorage	Browser	✅ Yes
sessionStorage	Browser	✅ Yes
HTTP/browser cache	Browser DevTools	✅ Partially inspectable

9. And here's a SUPER useful experiment for you

Open:

Chrome DevTools
→ Network
→ click the document request
→ Response



You'll be able to inspect the initial HTML response.

Then navigate with `<Link>` and inspect the subsequent requests.

You'll start seeing the difference between:

INITIAL LOAD

HTML
+
RSC information
+
Client JS



and:

CLIENT NAVIGATION

RSC-related response
+
existing browser document



That distinction is really important for App Router interviews.

10. One more thing: don't confuse "showing" with "executing"

This is the key sentence:

A browser can display the output of code without possessing or executing the code that generated that output.

For example:

Server Component:

```
<h1>{product.name}</h1>
  ↓
server knows product.name = "iPhone"
  ↓
result
  ↓
<h1>iPhone</h1>
  ↓
browser displays it
```

The browser doesn't need:

`</>` TypeScript

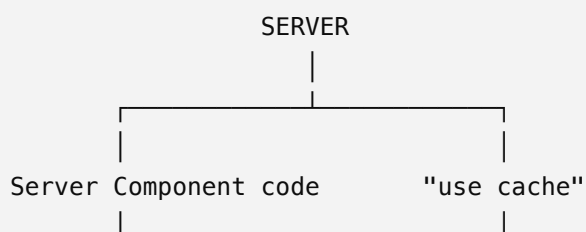
```
<h1>{product.name}</h1>
```

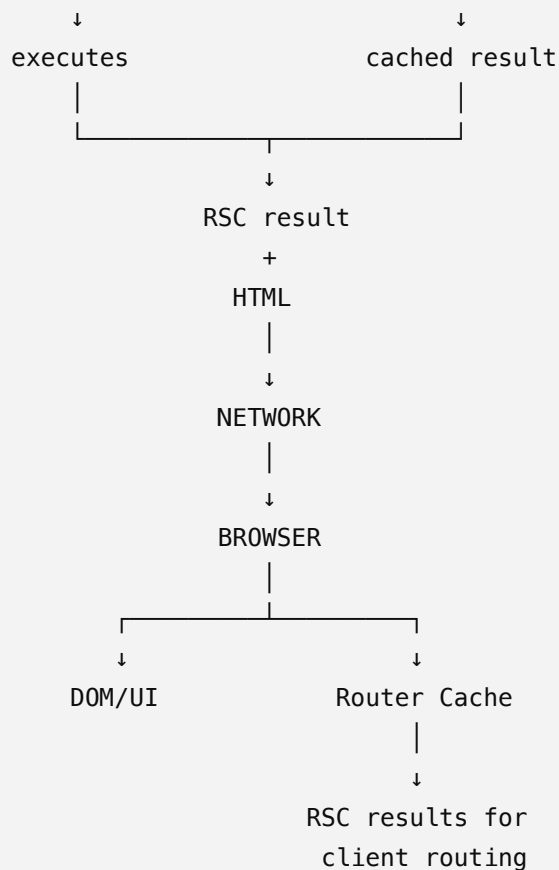
to be able to display:

iPhone

Final picture

Keep these two worlds separate:





🔥 The one thing to remember:

Server Component code stays on the server. Its *result* crosses the network.

And yes, **you can absolutely open DevTools and inspect the HTML and RSC network traffic.** You just generally can't open a little "Router Cache" folder and browse it like localStorage.

If you want, we can next do a **hands-on DevTools experiment** where I give you a tiny Next.js page and tell you **exactly what to click in Chrome DevTools and what you should expect to see.** That'll make this stick for real. 🙄



i willdo it after i build nextjs app ..next

Absolutely 🤪 We'll do the DevTools experiment **after you build the Next.js app** so you can actually see the HTML, RSC requests, navigation, and caching instead of just imagining them.

Next → Lesson 6.8 🚀

We'll move into:

Hydration vs Reconciliation vs RSC Updates

We'll answer:

Server Component renders on server

↓

HTML reaches browser

↓

Client Component JS arrives

↓

What exactly gets hydrated?

↓

What does React reconcile?

↓

What happens during router.refresh()?

↓

What happens during <Link> navigation?

And we'll use one small example throughout so the differences are crystal clear. 🧠



how any more?

Haha 🤔 you're asking **how many lessons are left in the whole Next.js roadmap**, right?

You completed **Lesson 4**, and we're currently around **Lesson 6.7** in the deeper caching/RSC section.

 Remaining roadmap

Lesson	Topic	Status
6.8	Hydration vs Reconciliation vs RSC Updates	➡️ SOON Next
6.9	Full request lifecycle — putting everything together	🕒
7	Next.js Performance	🕒
8	Authentication & Authorization	🕒

Lesson	Topic	Status
9	Middleware / Proxy & request interception	🕒
10	Route Handlers & APIs	🕒
11	Error Handling & Loading UX	🕒
12	Forms, Server Actions & Mutations — interview deep dive	🕒
13	SEO, Metadata & OG Images	🕒
14	Images, Fonts & Scripts	🕒
15	Deployment, Vercel, CDN & Edge	🕒
16	Testing Next.js apps	🕒
17	Security & production best practices	🕒
18	Next.js System Design	🕒
19	Interview questions + debugging scenarios	🕒
20	Build the Web Performance Analyzer 🚀	🕒

So roughly 14 major lessons/sections remain.

But don't worry — several of those are much smaller than the caching/RSC section we've just gone through. 😂

And 6.8 → 6.9 is basically the final deep architectural piece. After that we'll move into practical/interview-focused Next.js topics.

Your journey so far

- Lesson 1 → Server vs Client Components

Lesson 2 → App Router & Routing

Lesson 3 → Navigation & Routing details

Lesson 4 → Data Fetching

Lesson 5 → Mutations & Server Actions

Lesson 6 → Caching + Rendering architecture
 - └ Cache Components
 - └ use cache
 - └ cacheLife
- ✓

✓

✓

✓

✓

🔄

✓

✓

✓



└─ cacheTag	✓
└─ PPR	✓
└─ RSC	✓
└─ 6.8 Hydration/Reconciliation	→ SOON

So we're **well past the fundamentals now.** 🔥

And after the roadmap, we'll use all of this in your **Web Performance Analyzer project**, which is where the concepts should really become second nature.



ChatGPT can make mistakes. Check important info.